

Experiment No. 6

Implementation and Comparative Analysis of Image Segmentation Techniques using Python and OpenCV

Aim: To implement and evaluate Global Thresholding, Otsu's Thresholding, Adaptive Thresholding, Watershed Segmentation, and K-Means Clustering for image segmentation.

Libraries: OpenCV, NumPy, Matplotlib, Scikit-learn

This notebook is designed for **Google Colab**. It uses a built-in OpenCV sample image by default, so it can run without uploading a separate image. You can also upload your own image in the optional section.

```
# Install/import required libraries
!pip -q install opencv-python-headless scikit-learn

import cv2
import numpy as np
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import time

print("OpenCV:", cv2.__version__)
print("NumPy:", np.__version__)

OpenCV: 5.0.0
NumPy: 2.1.3
```

1. Load a real-world image

```
# Use OpenCV's built-in coins image as the default real-world image.
# This avoids requiring an external download and works directly in
# Colab.

url =
"https://raw.githubusercontent.com/opencv/opencv/master/samples/data/
sudoku.png"
import urllib.request

img_path = "/content/sudoku.png"
urllib.request.urlretrieve(url, img_path)

image = cv2.imread(img_path)
if image is None:
```

```

raise ValueError("Image could not be loaded.")

image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)
gray = cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)

plt.figure(figsize=(8,5))
plt.imshow(image_rgb)
plt.title("Original Image")
plt.axis("off")
plt.show()

print("Image shape:", image.shape)

```

Original Image



Image shape: (563, 558, 3)

2. Preprocessing using Grayscale Conversion and Gaussian Blur

```

blurred = cv2.GaussianBlur(gray, (5, 5), 0)

plt.figure(figsize=(12,4))
plt.subplot(1,2,1)

```

```
plt.imshow(gray, cmap="gray")
plt.title("Grayscale Image")
plt.axis("off")

plt.subplot(1,2,2)
plt.imshow(blurred, cmap="gray")
plt.title("Gaussian Blurred Image")
plt.axis("off")
plt.show()
```

Grayscale Image



Gaussian Blurred Image



3. Global Thresholding

```
start = time.perf_counter()

threshold_value = 127
_, global_thresh = cv2.threshold(blurred, threshold_value, 255,
cv2.THRESH_BINARY)

global_time = time.perf_counter() - start

plt.figure(figsize=(6,5))
plt.imshow(global_thresh, cmap="gray")
plt.title(f"Global Thresholding (T={threshold_value})")
plt.axis("off")
plt.show()

print(f"Execution time: {global_time*1000:.3f} ms")
```



Execution time: 6.070 ms

4. Otsu's Thresholding

```
start = time.perf_counter()

otsu_value, otsu_thresh = cv2.threshold(
    blurred, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
)

otsu_time = time.perf_counter() - start

plt.figure(figsize=(6,5))
plt.imshow(otsu_thresh, cmap="gray")
plt.title(f"Otsu's Thresholding (T={otsu_value:.0f})")
plt.axis("off")
plt.show()

print(f"Automatically selected threshold: {otsu_value:.0f}")
print(f"Execution time: {otsu_time*1000:.3f} ms")
```



Automatically selected threshold: 97
Execution time: 1.102 ms

5. Adaptive Thresholding

```
start = time.perf_counter()

adaptive_thresh = cv2.adaptiveThreshold(
    blurred, 255,
    cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
    cv2.THRESH_BINARY,
    11, 2
)

adaptive_time = time.perf_counter() - start

plt.figure(figsize=(6,5))
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Gaussian Thresholding")
plt.axis("off")
plt.show()

print(f"Execution time: {adaptive_time*1000:.3f} ms")
```

Adaptive Gaussian Thresholding



Execution time: 25.083 ms

6. Watershed Segmentation

```
# Watershed works best when foreground/background markers are prepared.  
# We use Otsu's result to create a binary mask, then derive sure foreground  
# and sure background regions.
```

```
binary = otsu_thresh.copy()
```

```
# Ensure foreground is white for the distance transform
```

```
if np.mean(binary) > 127:  
    binary = cv2.bitwise_not(binary)
```

```
kernel = np.ones((3,3), np.uint8)
```

```
opening = cv2.morphologyEx(binary, cv2.MORPH_OPEN, kernel,  
iterations=2)
```

```
sure_bg = cv2.dilate(opening, kernel, iterations=3)
```

```
dist_transform = cv2.distanceTransform(opening, cv2.DIST_L2, 5)  
_, sure_fg = cv2.threshold(
```

```
    dist_transform, 0.35 * dist_transform.max(), 255, 0
)
sure_fg = np.uint8(sure_fg)
unknown = cv2.subtract(sure_bg, sure_fg)

num_labels, markers = cv2.connectedComponents(sure_fg)
markers = markers + 1
markers[unknown == 255] = 0

watershed_image = image.copy()
start = time.perf_counter()
markers = cv2.watershed(watershed_image, markers)
watershed_time = time.perf_counter() - start

watershed_rgb = cv2.cvtColor(watershed_image, cv2.COLOR_BGR2RGB)
watershed_rgb[markers == -1] = [255, 0, 0]

plt.figure(figsize=(8,5))
plt.imshow(watershed_rgb)
plt.title("Watershed Segmentation (red = boundaries)")
plt.axis("off")
plt.show()

print("Number of initial connected components:", num_labels - 1)
print(f"Execution time: {watershed_time*1000:.3f} ms")
```

Watershed Segmentation (red = boundaries)



Number of initial connected components: 1
Execution time: 9.623 ms

7. K-Means Clustering for Color-Based Segmentation

```
# Resize for faster clustering while retaining visual quality.
small_rgb = cv2.resize(image_rgb, (0, 0), fx=0.5, fy=0.5)
pixels = small_rgb.reshape((-1, 3)).astype(np.float32)
```

```
K = 3
```

```
start = time.perf_counter()
```

```
kmeans = KMeans(n_clusters=K, random_state=42, n_init=10)
```

```
labels = kmeans.fit_predict(pixels)
```

```
centers = np.uint8(kmeans.cluster_centers_)
```

```
segmented = centers[labels].reshape(small_rgb.shape)
```

```
kmeans_time = time.perf_counter() - start
```

```
plt.figure(figsize=(8,5))
```

```
plt.imshow(segmented)
```

```
plt.title(f"K-Means Segmentation (K={K})")
```

```
plt.axis("off")
```

```
plt.show()

print("Cluster centers (RGB):")
print(centers)
print(f"Execution time: {kmeans_time*1000:.3f} ms")
```

K-Means Segmentation (K=3)



```
Cluster centers (RGB):
[[ 53  56  52]
 [134 139 135]
 [ 96 101  95]]
Execution time: 632.000 ms
```

8. Visual Comparison

```
plt.figure(figsize=(16,10))

plt.subplot(2,3,1)
plt.imshow(image_rgb)
plt.title("Original")
plt.axis("off")

plt.subplot(2,3,2)
plt.imshow(global_thresh, cmap="gray")
```

```

plt.title("Global Threshold")
plt.axis("off")

plt.subplot(2,3,3)
plt.imshow(otsu_thresh, cmap="gray")
plt.title("Otsu Threshold")
plt.axis("off")

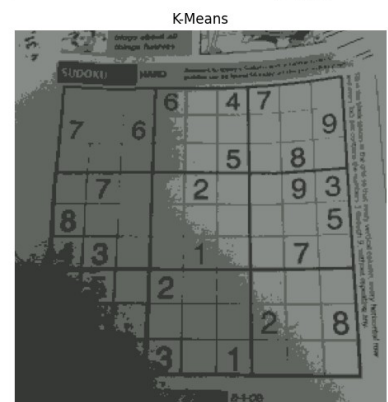
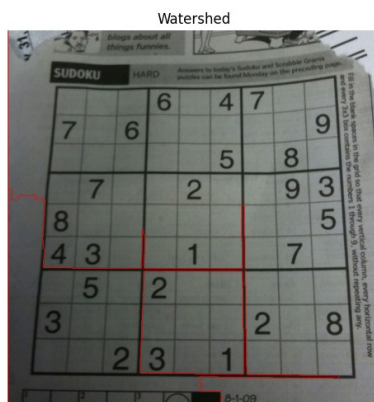
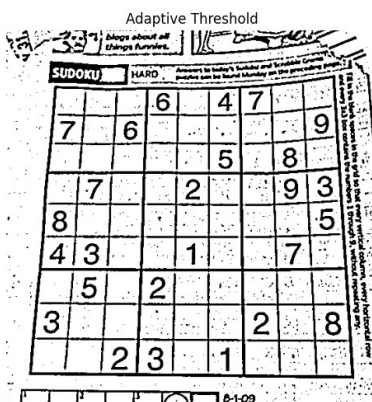
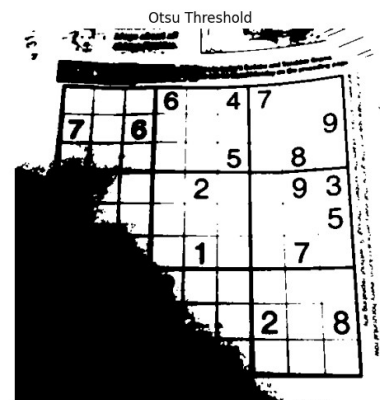
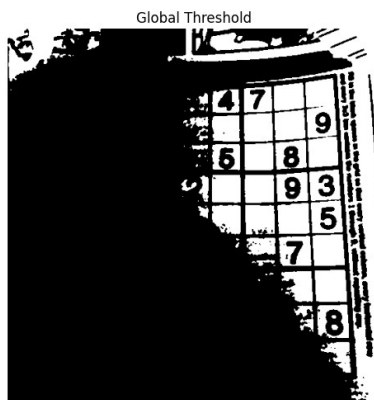
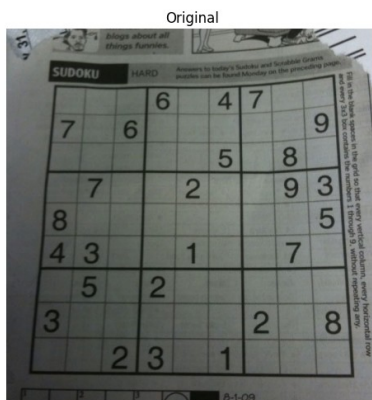
plt.subplot(2,3,4)
plt.imshow(adaptive_thresh, cmap="gray")
plt.title("Adaptive Threshold")
plt.axis("off")

plt.subplot(2,3,5)
plt.imshow(watershed_rgb)
plt.title("Watershed")
plt.axis("off")

plt.subplot(2,3,6)
plt.imshow(segmented)
plt.title("K-Means")
plt.axis("off")

plt.tight_layout()
plt.show()

```

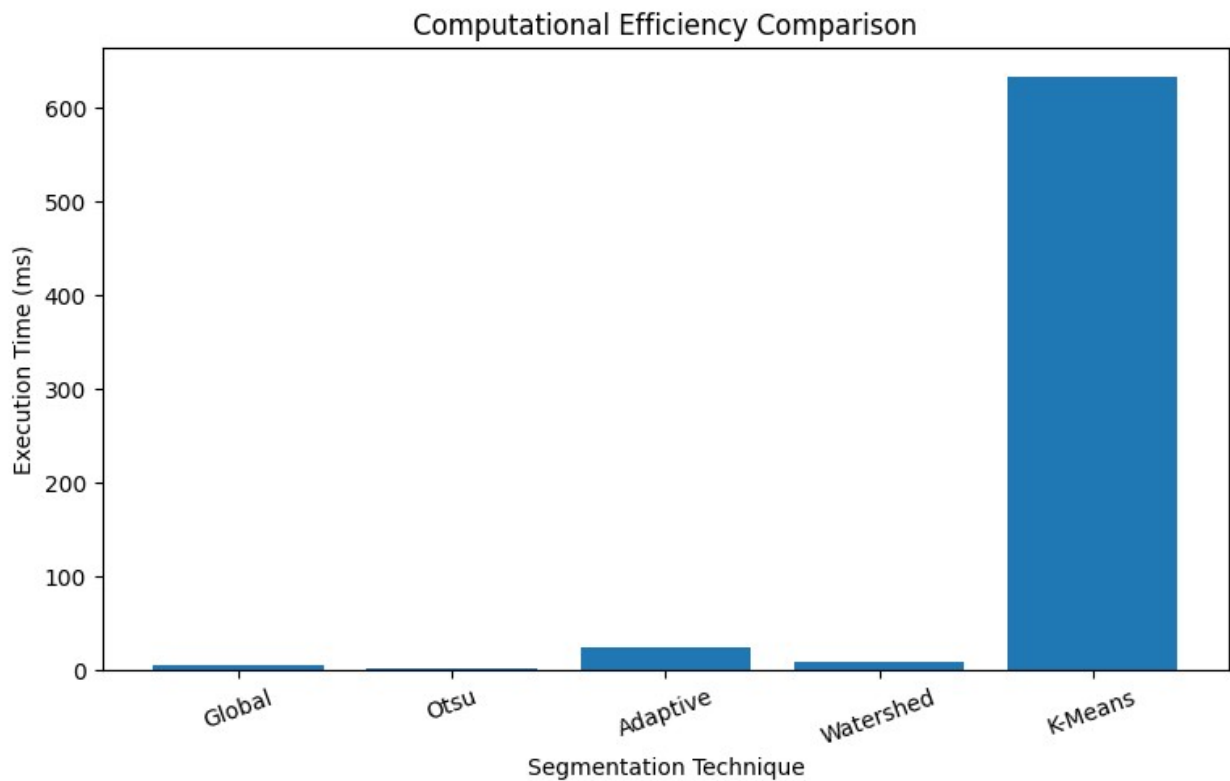


9. Comparison of Computational Efficiency

```
methods = ["Global", "Otsu", "Adaptive", "Watershed", "K-Means"]
times_ms = [
    global_time * 1000,
    otsu_time * 1000,
    adaptive_time * 1000,
    watershed_time * 1000,
    kmeans_time * 1000
]

plt.figure(figsize=(9,5))
plt.bar(methods, times_ms)
plt.ylabel("Execution Time (ms)")
plt.xlabel("Segmentation Technique")
plt.title("Computational Efficiency Comparison")
plt.xticks(rotation=20)
plt.show()

for method, t in zip(methods, times_ms):
    print(f"{method:12s}: {t:.3f} ms")
```



Global	: 6.070 ms
Otsu	: 1.102 ms

Adaptive	: 25.083 ms
Watershed	: 9.623 ms
K-Means	: 632.000 ms

10. Observations and Analysis

1. **Global Thresholding:** Simple and fast, but the fixed threshold may fail when illumination or contrast changes across the image.
2. **Otsu's Thresholding:** Automatically selects a threshold from the image histogram and is useful when foreground and background have reasonably distinct intensity distributions.
3. **Adaptive Thresholding:** Computes a local threshold for different neighborhoods, making it more suitable for uneven illumination.
4. **Watershed:** Produces region boundaries and is particularly useful for separating touching or overlapping objects when good markers are available.
5. **K-Means:** Groups pixels according to feature similarity (here, RGB color), making it useful for color-based segmentation. The number of clusters K must be selected.
6. **Efficiency:** Thresholding methods are generally computationally inexpensive. K-Means may require more computation because it iteratively updates cluster assignments and centers.
7. **Best technique:** There is no universally best method. The choice depends on image characteristics such as illumination, contrast, color, texture, noise, and object overlap.

Questions and Answers

1. What is image segmentation? Why is it a fundamental step in computer vision?

Image segmentation is the process of dividing an image into meaningful regions or objects. It is fundamental because it isolates regions of interest and makes later tasks such as object detection, recognition, measurement, tracking, and medical analysis easier and more accurate.

2. Differentiate between Image Segmentation and Image Classification.

Image classification assigns one or more labels to an entire image (or sometimes an input crop). **Image segmentation** assigns a class/region at the pixel level, producing a spatial map of objects or regions. For example, classification may say an image contains a car, while segmentation identifies exactly which pixels belong to the car.

3. Explain Global Thresholding, Otsu's Thresholding, and Adaptive Thresholding.

Global Thresholding: A single manually selected threshold T is applied to all pixels. Pixels above/below T are assigned to different classes.

Otsu's Thresholding: Automatically chooses a threshold by maximizing the separation between foreground and background intensity distributions, commonly by maximizing between-class variance.

Adaptive Thresholding: Calculates a threshold separately for local neighborhoods, so it works better when lighting is non-uniform.

4. What is the Watershed Algorithm? Why is it useful for separating overlapping objects?

Watershed treats an image or derived distance map as a topographic surface. Starting from markers, regions grow until boundaries meet. The resulting watershed lines can separate objects that touch or overlap, provided suitable markers are available.

5. How is K-Means Clustering applied to image segmentation?

Each pixel is represented by features such as RGB values. K-Means divides the pixels into K clusters by repeatedly assigning each pixel to the nearest cluster center and updating the centers. Pixels in the same cluster are given the same segment label.

6. Compare threshold-based and clustering-based segmentation.

Threshold-based methods use intensity cutoffs and are usually simple and fast, but can struggle with complex intensity distributions. Clustering methods group pixels using feature similarity and can handle multiple regions and colors, but require parameter selection such as K and can be computationally more expensive.

7. What challenges occur with complex backgrounds or varying illumination?

Challenges include noise, shadows, reflections, low contrast, texture, similar foreground/background colors, changing brightness, object overlap, and poorly defined boundaries. Local/adaptive methods, preprocessing, morphology, clustering, or learned models can help.

8. Mention five real-world applications.

1. Medical image analysis and tumor/organ segmentation
2. Autonomous driving and road-scene understanding
3. Satellite and remote-sensing image analysis
4. Industrial defect inspection
5. Face, person, and object segmentation in computer vision systems

9. How does segmentation improve object detection and image recognition?

Segmentation provides precise spatial information about objects and their boundaries. This can reduce background interference, improve localization, allow pixel-level measurements, and provide better inputs/features for downstream recognition and detection systems.

10. Compare traditional techniques with U-Net and Mask R-CNN.

Traditional techniques such as thresholding, watershed, and K-Means are relatively simple, interpretable, and often require little training data, but their performance can degrade on complex images. **U-Net** is a deep-learning architecture widely used for semantic segmentation and can learn complex visual patterns from annotated data. **Mask R-CNN** performs instance segmentation by detecting individual objects and predicting a separate mask for each instance. Deep-learning methods generally achieve better performance on difficult datasets but require labeled training data, greater computation, and model training/tuning.

Conclusion

The experiment successfully implemented five image segmentation techniques using Python and OpenCV. Global and Otsu thresholding provide simple intensity-based segmentation, adaptive thresholding handles uneven illumination, watershed is useful for separating touching objects, and K-Means provides color-based clustering. The comparative analysis shows that the most appropriate method depends on the characteristics of the input image and the requirements of the computer vision application.